

深圳大学《并行计算》期末冲刺综合模拟卷

PPT重点版 - 闭卷 - 120分钟 - 共10题 - 100分 - 参考答案

说明：以下为评分要点，不要求逐字一致；考场作答应优先写清概念、公式、错误原因和算法步骤。

第1题 参考答案与评分要点

1. Cache 按缓存行取数，访问一个元素时会把相邻若干元素一起载入。
2. C/C++ 二维数组行优先存储，按行访问地址连续，空间局部性好；按列访问跨行跳转，缓存行复用差，cache miss 更多。
3. UMA: 访问任意主存位置延迟基本相同，编程简单但扩展性受互连带宽限制。
4. NUMA: 本地内存快、远程内存慢，可扩展性较好但依赖数据局部性。
5. MPI 更偏分布式内存；Pthreads/OpenMP 更偏共享内存；NUMA 仍可表现为共享地址空间，但访问时间不均匀。

第2题 参考答案与评分要点

1. 固定负载: $S=T1/Tp=N/(N/p+5)=Np/(N+5p)$, $E=S/p=N/(N+5p)$ 。p 趋于无穷时 S 趋于 N/5，存在上限。
2. 固定时间: 令 $N/p+5=T$, 得 $N=p(T-5)$ 。同规模串行时间 $T1'=p(T-5)$, $S=T1'/T=p(T-5)/T=p(1-5/T)$ 。当 T 远大于 5 时, S 近似 p。
3. Torus 每维取较短距离。x 方向 $\min(5,3)=3$, y 方向 $\min(6,2)=2$, 总距离 5。
4. 一条路径: $(1,1) \rightarrow (0,1) \rightarrow (7,1) \rightarrow (6,1) \rightarrow (6,0) \rightarrow (6,7)$ 。

第3题 参考答案与评分要点

1. 发送字符串应发送 $\text{strlen(msg)}+1$, 包含结尾 '\0', 否则接收端按字符串输出可能越界或乱码。
2. 接收数据类型应为 MPI_CHAR, 不是 MPI_INT。
3. 发送 tag 为 0, 接收 tag 为 1, 消息不匹配; 应统一 tag, 或接收端使用 MPI_ANY_TAG。
4. 应确保至少有 2 个进程; 实际程序中可检查 MPI_Comm_size。
5. status 需要声明 MPI_Status status; 若不需要可用 MPI_STATUS_IGNORE。
6. 计时应使用 MPI_Wtime 得到墙钟时间, 通常在计时前后加 MPI_Barrier, 并取各进程 elapsed 最大值。
7. clock 测的是进程 CPU 时间, 不能代表 MPI 程序实际经过时间; MPI_Wtime 包括通信、等待和同步开销。

第4题 参考答案与评分要点

1. 将 A、B、C 划分为 $q \times q$ 个块, 处理器 $P(i,j)$ 负责 $C(i,j)$ 的块结果。
2. 初始对准: A 的第 i 行块循环左移 i 步, B 的第 j 列块循环上移 j 步。
3. 每轮先本地计算 $C(i,j) += A_block * B_block$, 然后 A 块向左循环移动一格, B 块向上循环移动一格。
4. 共执行 q 轮, $P(i,j)$ 最终得到 $C(i,j) = \sum_{k=0}^{q-1} A(i,k)B(k,j)$ 。
5. 通信发生在处理器行和列的环形邻居之间, 可用 MPI_Sendrecv 避免死锁。

第5题 参考答案与评分要点

1. 代码思想: 用 counter 统计到达 barrier 的线程数, 最后一个线程 broadcast 唤醒其余线程。
2. 不可靠可复用: 没有 generation/phase 区分第几轮 barrier; counter 清零后可能与下一轮混淆。
3. 条件变量可能虚假唤醒; 被唤醒只说明条件可能改变, 不代表条件一定满足。
4. pthread_cond_wait 应写在 while 循环里, 醒来后重新检查共享条件。
5. 改进: 增加 generation, 线程记录 my_generation; 最后一个线程 counter=0、generation++、broadcast; 其他线程 while(my_generation!=generation) wait。

第6题 参考答案与评分要点

1. 伪共享: 多个线程修改不同变量, 但这些变量落在同一缓存行, 导致缓存一致性协议反复使缓存行失效。
2. 它不是逻辑数据竞争, 但会造成频繁缓存行迁移和失效, 降低性能。
3. 优化 1: padding, 让不同线程频繁写入的数据落在不同缓存行。
4. 优化 2: 每个线程先用私有临时变量或私有数组计算, 最后集中写回共享结果。
5. 读写锁适合读多写少场景: 多读者可并发, 写者独占。
6. 写者饥饿: 如果新读者不断获得读锁, 等待写锁的线程长期无法执行。可用公平或写者优先策略缓解。

第7题 参考答案与评分要点

1. sum 被多个线程同时更新, 存在 race condition。
2. factor 在原代码中为共享变量, 也会被多个线程覆盖; 应声明为循环内局部变量或 private。
3. 推荐写法: #pragma omp parallel for num_threads(10) reduction(+:sum), 循环内 double factor = ...。
4. reduction 为每线程创建 sum 的私有副本, 最后合并, 最适合本题。
5. critical 或 atomic 也可保护 sum 更新, 但会频繁串行化, 性能较差; atomic 不适合保护复杂代码块。
6. 浮点归约改变加法顺序, 结果可能和串行略有差异, 不一定是程序错误。

```
double sum = 0.0;
#pragma omp parallel for num_threads(10) reduction(+:sum)
for (long i = 0; i < n; i++) {
    double factor = (i % 2 == 0) ? 1.0 : -1.0;
    sum += factor / (2 * i + 1);
}
double pi = 4.0 * sum;
```

第8题 参考答案与评分要点

1. PSRS 步骤: 均匀划分、局部排序、正则采样、样本排序、选择主元、按主元分桶、全局交换/多路归并。
2. p=3 时每段 6 个元素。P0 排序: 4,6,8,10,14,16; P1 排序: 5,11,12,13,17,18; P2 排序: 1,2,3,7,9,15。
3. 正则采样取下标 0,2,4: P0 取 4,8,14; P1 取 5,12,17; P2 取 1,3,9。
4. 样本排序: 1,3,4,5,8,9,12,14,17。本题约定取下标 p 和 2p, 即 5 和 12 为主元。
5. 按主元分桶: ≤ 5 、 $(5,12]$ 、 >12 , 各处理器把对应桶发送给负责该区间的处理器, 最后各自多路归并。
6. OpenMP task 归并排序: parallel 创建线程组, single 保证只有一个线程启动最外层递归, 避免重复创建整棵任务树; 左右递归创建 task, merge 前必须 taskwait 等待子任务完成。

第9题 参考答案与评分要点

1. grid 是一次 kernel 启动形成的线程网格；block 是线程块；thread 是执行单元；warp 通常为 32 个线程的硬件调度单位；SM 是执行 block 的流多处理器。
2. 矩阵二维映射通常写作 $\text{row} = \text{blockIdx.y} * \text{blockDim.y} + \text{threadIdx.y}$ ， $\text{col} = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$ 。
3. 原代码把 x 维当 row、y 维当 col，和常见行列映射相反，容易导致访存和配置理解错误。
4. 必须加边界判断 $\text{if}(\text{row} < n \ \&\& \ \text{col} < n)$ ，因为 grid 尺寸通常向上取整，多余线程可能越界。
5. block 被调度到 SM 上执行，一个 block 不会拆到多个 SM；warp 是调度单位，不是用户显式创建的 block。

```
__global__ void MatMul(double* A, double* B, double* C, int n) {  
    int row = blockIdx.y * blockDim.y + threadIdx.y;  
    int col = blockIdx.x * blockDim.x + threadIdx.x;  
    if (row < n && col < n) {  
        double sum = 0.0;  
        for (int k = 0; k < n; k++) {  
            sum += A[row * n + k] * B[k * n + col];  
        }  
        C[row * n + col] = sum;  
    }  
}
```

第10题 参考答案与评分要点

1. shared memory 是同一 block 内线程共享的片上存储，可保存局部和，减少全局内存访问。
2. warp shuffle 可在同一 warp 内直接交换寄存器值，适合做 warp 内快速归约。
3. 多 warp block 常先在各 warp 内归约得到 warp_sum，再把各 warp_sum 放入 shared memory，由一个 warp 做第二级归约。
4. __syncthreads 只同步同一个 block 内线程，不能同步不同 block，因为不同 block 的调度顺序和驻留时间不由单个 block 控制。
5. 跨 block 全局同步通常拆成多个 kernel，或使用支持全局同步的专门机制；普通 kernel 内不要假设 block 间同步。
6. Bitonic sort 核心：先通过比较交换构造双调序列，再按阶段逐步做双调归并；每一级比较器把一对元素按方向交换，最终得到全局有序序列。